AI 388 / CS 388 Natural Language Processing

Module 4: Language Modeling and Self-Attention

Topics 12, 13 (+ perplexity)

Fall 2026 midterm study guide

Built from the lecture videos/transcripts, slides, Canvas exercises, Assignments 1–3 and the Fall 2022/2023 midterms.

Prepared October 2026 · lecture notation throughout

How to use this guide

Each **Topic** number is the item number on the official Fall 2026 midterm topic list. Section numbers are *topic.section*, so "2.4" is Topic 2, Section 4. Every topic has the same layout:

- **In plain words**: the idea in one paragraph.
- **Key boxes**: definitions and formulas in **lecture notation**.
- **Steps**: the algorithm or exam procedure, step by step.
- **Worked examples**: full calculations (checked numerically with Python).
- **Past exam**: every relevant Fall 2022/2023 midterm question and Canvas exercise, with the official answer and an explanation. Official errata are noted.
- **Traps**: places where convention differences or common slips cost points.
- **Practice**: original questions, with answers right after.

Notation (lecture): input $\mathbf{x}$; feature vector $f(\mathbf{x}) \in \mathbb{R}^d$; weights $\mathbf{w}$; label $y \in \{-1, +1\}$ (binary) or $y \in \mathcal{Y}$ (multiclass); step size α; D training examples $(\mathbf{x}^{(i)}, y^{(i)})$; log is the natural log unless a question says otherwise. Perceptron decision rule: positive iff $\mathbf{w}^\top f(\mathbf{x}) > 0$ (a score of 0 is negative). Where a past exam used a different convention, the box says so in red.

Exam reminders. Auto-graded short answers must match the requested format exactly (rounding, fractions vs. decimals, list syntax like `[w1, w2]`). Adding explanation text makes the answer wrong. Long answers: show work concisely for partial credit and box final answers.

Contents

How to use this guide **1**

12 n-gram Language Modeling (+ Smoothing, Perplexity, Neural LMs) **3**

12.1 Definition . . . 3
12.2 Uses (lecture) . . . 4
12.3 Smoothing: absolute discounting . . . 4
12.4 Evaluation: perplexity . . . 5
12.5 Neural LMs and RNNs (background for Topic 13, Topic 15) . . . 5
12.6 Practice . . . 6

13 Self-Attention: Definition and Mechanics **7**

13.1 Attention with one query (lecture 4.2.0) . . . 7
13.2 Self-attention (lecture 4.2.1) . . . 7
13.3 Multi-head self-attention (lecture 4.2.2) . . . 9
13.4 Positional encodings (lecture 4.2.3) . . . 9
13.5 Practice . . . 10

TOPIC 12

n-gram Language Modeling (+ Smoothing, Perplexity, Neural LMs)

Sources: Lectures 4.0.0 n-gram LMs, 4.0.1 Smoothing in n-gram LMs, 4.0.2 LM Evaluation, 4.1.0 Neural Language Models, 4.1.1 RNNs and their Shortcomings; slides seg-41, seg-42, seg-lm3, seg-43, seg-lm5; Exercise 4.0; Fall 2023 Q1(5), Q1(7–8).

In plain words

A language model (LM) assigns a probability to every sentence. By the chain rule, that's the product of each word's probability given all previous words. An **n-gram LM** simplifies by looking only at the previous $n-1$ words. Each conditional probability is then just "how often did this word follow this context in a big corpus?": **count and normalize**. Unseen combinations get probability 0, so we **smooth** by shaving a little mass off seen events and backing off to shorter contexts. LMs are evaluated by **perplexity**: exp of the average negative log-probability on held-out text. Lower is better.

12.1 Definition

Chain rule and the n-gram assumption

$$P(w_1,\dots,w_m) = \prod_{i=1}^{m} P(w_i \mid w_1,\dots,w_{i-1}) \quad \text{(exact)} \qquad \approx \qquad \prod_{i=1}^{m} P(w_i \mid w_{i-n+1},\dots,w_{i-1}) \quad (n\text{-gram}).$$

Bigram ($n=2$): $P(\mathbf{w}) = P(w_1|\langle s\rangle)P(w_2|w_1)P(w_3|w_2)\cdots$ **Trigram**: $P(w_1|\langle s\rangle,\langle s\rangle)P(w_2|\langle s\rangle,w_1)P(w_3|w_1,w_2)\cdots$ ($n-1$ start symbols). An n-gram LM is an $(n-1)$**-order Markov model** ($w_3 \perp w_1 \mid w_2$ in a bigram LM), like HMM tag transitions (Topic 20). Typical n: 3–7 (5-grams common).

Parameters and estimation (count and normalize)

Multinomial distributions: a bigram model has a $|V| \times |V|$ table (one distribution over next words per context word). Maximum-likelihood estimate:

$$P(w \mid \text{ctx}) = \frac{\text{count}(\text{ctx}, w)}{\text{count}(\text{ctx})}, \qquad \text{e.g. } P(\text{dog}|\text{the}) = \frac{c(\text{the dog})}{c(\text{the})}.$$

Distributions are typically *flat* (after *the*, almost anything can follow). Like skip-gram, it's an under-constrained prediction problem. Whether a stop symbol $\langle/s\rangle$ is generated depends on the setup. Follow the question (Fall 2023 Q7 has no stop symbol).

Worked example: bigram estimation and sentence probability

Corpus: "$\langle s\rangle$ I am Sam $\langle/s\rangle$", "$\langle s\rangle$ Sam I am $\langle/s\rangle$", "$\langle s\rangle$ I do not like green eggs and ham $\langle/s\rangle$".

- $P(\text{I}|\langle s\rangle) = 2/3$, $P(\text{Sam}|\langle s\rangle) = 1/3$, $P(\text{am}|\text{I}) = 2/3$, $P(\text{do}|\text{I}) = 1/3$, $P(\text{Sam}|\text{am}) = 1/2$,

$P(\langle/s\rangle|\text{am}) = 1/2$, $P(\langle/s\rangle|\text{Sam}) = 1/2$.

- $P(\langle s\rangle\ \text{I am Sam}\ \langle/s\rangle) = \frac{2}{3}\cdot\frac{2}{3}\cdot\frac{1}{2}\cdot\frac{1}{2} = \frac{1}{9} \approx 0.111$.
- Perplexity of this sentence over its 4 predictions: $(1/9)^{-1/4} = \sqrt{3} \approx 1.732$.
- $P(\text{I am ham})$: $c(\text{am ham}) = 0$, so the whole product is 0. That's why we smooth.

Midterm Fall 2023 Q1(7)–(8)

Bigram counts (a, b) = times b followed a: (the, frog) 4; (frog, is) 2; (frog, was) 4; (is, UNK) 1; (is, frog) 1; (UNK, UNK) 0. Vocabulary {the, frog, is, was}; everything else maps to UNK; $P(\text{the}|\langle s\rangle) = 1$.
(7) $P(\text{the frog is blue})$, no smoothing: $1\cdot\frac{4}{4}\cdot\frac{2}{6}\cdot\frac{1}{2} =$

Official answer: D. $1/6$ (blue→UNK; frog was followed 6 times; is 2 times).
(8) Which added sentence increases this probability most?

Official answer: A. "the frog is black": $1\cdot\frac{5}{5}\cdot\frac{3}{7}\cdot\frac{2}{3} = \frac{2}{7} \approx 0.286$. Check the others: B "frog is frog is frog is" gives $\frac{5}{9}\cdot\frac{1}{4} = \frac{5}{36} \approx 0.139$; C "the is blue" gives $\frac{4}{5}\cdot\frac{1}{3}\cdot\frac{2}{3} = \frac{8}{45} \approx 0.178$; D "frog frog frog frog" gives $\frac{2}{9}\cdot\frac{1}{2} = \frac{1}{9}$. **Method:** add the new bigram counts, recompute *both* numerators and context totals.

12.2 Uses (lecture)

Phrase-based MT (score how well stitched phrases read in the target language); grammatical error correction (does swapping an article raise the LM probability?); and, generally, LMs as "word2vec++": next-word prediction forces learning rich representations (pre-training, Topic 16).

12.3 Smoothing: absolute discounting

Absolute discounting with backoff (lecture form)

$$P_{AD}(w \mid \text{ctx}) = \frac{\max(c(\text{ctx}, w) - k,\, 0)}{c(\text{ctx})} + \lambda(\text{ctx})\, P_{AD}(w \mid \text{shorter ctx}), \qquad 0 < k < 1,$$

$$\lambda(\text{ctx}) = \frac{k \cdot \#\{\text{distinct word types seen after ctx}\}}{c(\text{ctx})} \quad \text{(exactly the mass removed, so it normalizes).}$$

Recurse down to unigrams, so any word seen in training gets nonzero probability. **Kneser–Ney** (the standard): same framework, but the lower-order distribution uses the *fertility* of contexts (how many distinct words a context is followed by), not raw counts.

Worked example: lecture: "want to go to ___" ($k = 0.2$)

Counts: Maui 2, class 1, campus 1 (total 4, 3 types). Discounted: Maui $1.8/4 = 0.45$, class $0.8/4 = 0.2$, campus 0.2. Reserved $\lambda = 3(0.2)/4 = 0.6/4 = 0.15$. $P(\text{Austin}|\text{want to go to}) = 0 + 0.15 \cdot P_{AD}(\text{Austin}|\text{to go to})$, which is nonzero once the backoff reaches a context where Austin was seen. Why more context helps: "want to go to" vs. "hate to go to" predict very different next words (Austin vs. class), but longer contexts are sparse.

12.4 Evaluation: perplexity

Perplexity (lecture 4.0.2)

Accuracy makes little sense (next-word prediction is "impossible"). Use the log-likelihood of **held-out** data, averaged per token:

$$\text{PPL} = \exp\Big(-\frac{1}{N}\sum_{i=1}^{N}\log P(w_i \mid w_{<i})\Big) = \Big(\prod_{i=1}^{N} P(w_i|w_{<i})\Big)^{-1/N}.$$

Lower is better. Interpretation: the average *branching factor*, i.e. the geometric mean of $1/P$. Example: probabilities $\frac{1}{4}, \frac{1}{3}, \frac{1}{4}, \frac{1}{3}$ give PPL $= (4 \cdot 3 \cdot 4 \cdot 3)^{1/4} = \sqrt{12} = 3.464$. A uniform model over $|V|$ words has PPL $= |V|$. Typical values range from about 10 to 200 depending on the domain. The base doesn't matter as long as exp and log match ($2^{-\frac{1}{N}\sum \log_2 p}$ is the same number).

Calculator recipe: perplexity

(1) Get each token's $P(w_i|\text{context})$. (2) Sum ln P. (3) Divide by $-N$ (number of predicted tokens). (4) e^x. Shortcut: PPL $= (\prod p_i)^{-1/N}$. If you have the average NLL (the A3 loss in nats), PPL $= e^{\text{avg NLL}}$.

12.5 Neural LMs and RNNs (background for Topic 13, Topic 15)

- Neural LM: $P(y|w_{1:i-1}) = \text{softmax}\big(\mathbf{u}_y \cdot f(w_{1:i-1})\big)$, where f embeds the context.
- f = **DAN**: ignores order (but the previous word matters far more than one 15 words back). f = **FFNN over concatenated embeddings** (Bengio et al. 2003): parameters grow with context length, and position-specific weights mean "great" in slot 4 shares nothing with "great" in slot 3.
- **RNN:** a cell updates state $\mathbf{h}_t = \tanh(W\mathbf{x}_t + V\mathbf{h}_{t-1} + \mathbf{b})$, sharing parameters across positions and handling arbitrary length.
- **RNN shortcomings:** (1) *vanishing gradients*: saturated tanh, and repeated multiplication by V makes gradients shrink or explode; even LSTMs struggle over 1000-token spans. (2) *Slow*: $O(n)$ sequential steps that can't be parallelized. (3) The state is dominated by recent tokens (Fall 2023 Q1(5)). Transformers fix (1) and (2) with self-attention.

Canvas Exercise 4.0

Q1: a DAN predicting "baseball players play ___" (after "Basketball players play basketball. Tennis players play tennis."). Why might it fail?

Answer: It struggles to encode relations/structure between words, and it can't encode positional information. (It *can* see far-back words.)

Q2: an FFNN over the last 4 words for "Alice bought a pair of sneakers. … she realized they didn't ___".

Answer: It can't look at critical words far back in the sentence.

Traps

- The context count is $c(\text{ctx})$ = how many times ctx appeared *as a context* (followed by something). It isn't necessarily the raw unigram count if the word can end a sentence without a stop symbol.

- One zero factor makes the whole sentence probability 0 under MLE.
- Map OOV words to UNK *before* counting and before scoring.
- Perplexity: divide by the number of *predicted* tokens (include $\langle/s\rangle$ if it's predicted; don't count $\langle s\rangle$).
- Lower perplexity is better; higher log-likelihood is better.

12.6 Practice

Practice

1. Using the Sam corpus, compute $P(\langle s\rangle \text{ Sam I am } \langle/s\rangle)$ under the bigram MLE.
2. A model assigns probabilities 0.5, 0.25, 0.125, 0.5 to a 4-token held-out text. Compute the perplexity.
3. Absolute discounting with $k = 0.5$: the context "I like" was seen 10 times, followed by 4 distinct words, one of them ("pizza") 5 times. Give the discounted term for pizza and λ.
4. Why is the perplexity of a uniform distribution over $|V|$ words equal to $|V|$?

Answers

1. $\frac{1}{3} \cdot P(\text{I}|\text{Sam}) \cdot \frac{2}{3} \cdot \frac{1}{2}$. $P(\text{I}|\text{Sam}) = c(\text{Sam I})/c(\text{Sam}) = 1/2$, so $\frac{1}{3} \cdot \frac{1}{2} \cdot \frac{2}{3} \cdot \frac{1}{2} = \frac{1}{18}$.
2. Product $= 0.5 \cdot 0.25 \cdot 0.125 \cdot 0.5 = 0.0078125 = 2^{-7}$, so PPL $= 2^{7/4} \approx 3.364$.
3. $(5 - 0.5)/10 = 0.45$; $\lambda = 0.5 \cdot 4/10 = 0.2$.
4. Each $p = 1/|V|$, so the geometric mean of $1/p$ is $|V|$.

TOPIC 13

Self-Attention: Definition and Mechanics

Sources: Lectures 4.2.0 Attention, 4.2.1 Self-Attention, 4.2.2 Multi-Head Self-Attention, 4.2.3 Position Encodings; slides seg-lm6, seg-lm7, seg-lm8, seg-lm9; Exercises 4.1, 4.2; A3 Part 1; Fall 2023 Q1(13).

In plain words

Attention is a soft lookup. A *query* says what you're looking for, each position offers a *key* saying what it is, and you score every key against the query (dot product). A softmax turns the scores into weights, and you return a weighted average of the positions' *values*. **Self-attention** does this for every position at once: each token is simultaneously a query, a key and a value. The output is a new vector per token that mixes in information from the tokens it attended to, a *contextualized* representation. Everything is parallel, and any token can reach any other in one step.

13.1 Attention with one query (lecture 4.2.0)

Three steps

1. **Scores:** $s_i = \mathbf{k}_i \cdot \mathbf{q}$ for each key i.
2. **Softmax:** $\alpha_i = \exp(s_i) / \sum_j \exp(s_j)$.
3. **Output:** $\mathbf{o} = \sum_i \alpha_i \, \mathbf{e}_i$ (weighted sum of the inputs, later of the *values*).

Worked example: lecture: does the sequence contain a `b`? ($e = 3$)

Embeddings $a = [1,0]$, $b = [0,1]$. Sequence a a b a, so keys = embeddings. Query $\mathbf{q} = [0,1]$ ("look for b").

1. Scores: $0, 0, 1, 0$.
2. Softmax with $e = 3$: $(1,1,3,1)/6 = (\frac{1}{6}, \frac{1}{6}, \frac{1}{2}, \frac{1}{6})$.
3. Output: $\frac{1}{6}[1,0] \cdot 3 + \frac{1}{2}[0,1] = [\frac{1}{2}, \frac{1}{2}]$. A plain average would give $[\frac{3}{4}, \frac{1}{4}]$, so attention amplified the "b-ness".

Sharper attention: multiply keys by $W^K = 10I$. Scores become $0, 0, 10, 0$, so weights are $\left[\frac{1}{e^{10}+3}, \frac{1}{e^{10}+3}, \frac{e^{10}}{e^{10}+3}, \frac{1}{e^{10}+3}\right]$ (**Exercise 4.1** answer), $\approx (0, 0, 1, 0)$.

13.2 Self-attention (lecture 4.2.1)

Scaled dot-product self-attention (Vaswani et al. 2017), one head

Input $X \in \mathbb{R}^{n \times d}$ (one row per token). Parameters $W^Q, W^K \in \mathbb{R}^{d \times d_k}$, $W^V \in \mathbb{R}^{d \times d_v}$.

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V, \qquad \text{Attention}(Q,K,V) = \underbrace{\text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)}_{A \in \mathbb{R}^{n \times n}} V \in \mathbb{R}^{n \times d_v}.$$

- $S = QK^\top$: $S_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j$, the compatibility of query i with key j.

- Softmax is **row-wise**: row i of A is a probability distribution over the n positions that token i attends to. That's n distributions (Exercise 4.2 Q1).
- Output row $i = \sum_j A_{ij}\mathbf{v}_j$, a weighted combination of the rows of V.
- $\sqrt{d_k}$ scaling keeps dot products from growing with dimension, which would make the softmax overly peaked (lecture: "not too important").
- "API": a sequence of n vectors in, a sequence of n contextualized vectors out.

Worked example: lecture's matrix version

E = rows [1,0],[1,0],[0,1],[1,0] (a a b a). $W^Q = \begin{bmatrix} 0 & 1 \\ 0 & 1 \end{bmatrix}$, so every row of $Q = EW^Q$ is [0,1] (every token looks for b). $W^K = 10I$, so $K = 10E$. $S = QK^\top$ has every row $= [0, 0, 10, 0]$. Row-wise softmax puts ≈ 1 on position 3 for every token. Output $= A\,EW^V$. Many different parameterizations give the same behaviour; learned weights won't look this clean.

Worked example: full numeric self-attention (verified)

$X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$, $W^Q = I$, $W^K = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$, $W^V = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}$, $d_k = 2$.

1. $Q = X$; $K = XW^K = \begin{bmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 2 \end{bmatrix}$; $V = XW^V = \begin{bmatrix} 1 & 0 \\ 0 & 2 \\ 1 & 2 \end{bmatrix}$.
2. $S = QK^\top = \begin{bmatrix} 1 & 0 & 1 \\ 1 & 1 & 2 \\ 2 & 1 & 3 \end{bmatrix}$; scaled $S/\sqrt{2} = \begin{bmatrix} 0.707 & 0 & 0.707 \\ 0.707 & 0.707 & 1.414 \\ 1.414 & 0.707 & 2.121 \end{bmatrix}$.
3. Row-wise softmax: $A = \begin{bmatrix} 0.401 & 0.198 & 0.401 \\ 0.248 & 0.248 & 0.504 \\ 0.284 & 0.140 & 0.576 \end{bmatrix}$ (each row sums to 1).
4. Output $AV = \begin{bmatrix} 0.802 & 1.198 \\ 0.752 & 1.504 \\ 0.860 & 1.432 \end{bmatrix}$.

Without the $\sqrt{d_k}$ scaling, row 1 of A would be $(0.422, 0.155, 0.422)$, which is peakier.

Shapes and cost

$X : n \times d$; $Q, K : n \times d_k$; $V : n \times d_v$; $S, A : \mathbf{n} \times \mathbf{n}$; output $n \times d_v$. Batched: prepend the batch dimension, e.g. A is [batch, n, n]. Cost (Exercise 4.2 Q2): computing Q, K, V is $O(nd\,d_k)$; $QK^\top$ is $O(n^2 d_k)$. Total $O(nk_1k_2 + n^2k_2)$ for input size k_1, key size k_2. **Quadratic in sequence length** (memory too) is the main drawback. **Pros vs. RNNs:** no sequential operations (fully parallel), and path length 1 between any two tokens.

13.3 Multi-head self-attention (lecture 4.2.2)

MHSA

One head's softmax tends to be **peaky**, so it can't easily blend two things ("I visited New __" needs both *visited* and *New*). Run h independent heads, each with its own W_i^Q, W_i^K, W_i^V (randomly initialized, so they learn different patterns):

$$\text{head}_i = \text{softmax}\left(\frac{XW_i^Q(XW_i^K)^\top}{\sqrt{d_k}}\right)XW_i^V, \qquad \text{MHSA}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W^O.$$

$W^O \in \mathbb{R}^{hd_v \times d}$ projects back to the model dimension, so input and output are both $n \times d$. Illustrated Transformer example: 8 heads with $d_v = 3$, concat gives 24 dims, W^O brings it back down. Standard choice: $d_k = d_v = d/h$. Parameters per MHSA layer (no biases): $3d \cdot (hd_k) + hd_v \cdot d = 4d^2$ when $hd_k = d$.

13.4 Positional encodings (lecture 4.2.3)

Self-attention is **permutation-equivariant**: without position information, a query can't tell where tokens are ("visited" and "ate" look the same from the blank's perspective).

scheme	how
Absolute (learned)	A second embedding table indexed by position $(1, 2, 3, \ldots)$; **add** it to the word embedding at the input. Simple and standard (GPT-3; A3). Drawback: no embedding exists for positions beyond the maximum trained length, and nothing is shared between positions.
Sinusoidal (Vaswani)	Fixed (not learned) sines/cosines at different frequencies, added to the input. Nearby positions get similar vectors (only high-frequency dims differ); distant ones differ more.
Relative (T5)	Inject a learned function of the *distance* $i - j$ directly into the $QK^\top$ scores.
ALiBi	Add $-m \cdot (\text{distance})$ to the scores: a fixed linear penalty with a different slope m per head (some heads local, some global). Very few parameters.
NoPE (2023)	No positional encoding at all. Works for *causal* (past-only) attention, which can count tokens implicitly. Fails for bidirectional attention.

Midterm Fall 2023 Q1(13) (A3 language model)

Possible shapes for the attention map?

Official answer: II. [seq len, seq len]; III. [batch size, seq len, seq len]. Not [batch, seq] (that's one score per token), not [batch, seq, d_{model}] (that's the output), not $d_k \times d_k$ or $d_{model} \times d_{model}$ (those are parameter shapes).

Canvas Exercise 4.2

Q1: number of probability distributions for a length-n sentence:

Answer: n (n^2 values in total). Q2: complexity of the self-attention matrix:

Answer: $O(nk_1k_2 + n^2k_2)$.

Traps

- Softmax over **rows** (over keys j for a fixed query i), not columns.
- A is $n \times n$, independent of d. The output has the value dimension.
- Attention weights are *activations*, not parameters. The parameters are W^Q, W^K, W^V (and W^O).
- Positional encodings are *added* to the embeddings (same dimension), not concatenated, in the lecture/A3 scheme.
- Self-attention alone isn't a Transformer: you also need the FFN, residuals and layer norm (Topic 14).
- If a question gives $e = 3$, use it in the softmax.

13.5 Practice

Practice

1. Keys $[1,0], [0,1], [1,1]$, values = keys, query $[2,0]$, no scaling, e real. Compute the attention weights and output.
2. $n = 10$ tokens, $d = 64$, 4 heads, $d_k = d_v = 16$. Give the shapes of each head's Q, A, head output, the concatenation, and W^O.
3. Apply a causal mask to the full numeric example (each token sees only itself and earlier tokens). Give row 2 of A and row 2 of the output.

Answers

1. Scores $2, 0, 2$: weights $(e^2, 1, e^2)/(2e^2+1) = (0.4683, 0.0634, 0.4683)$. Output $= 0.4683[1,0] + 0.0634[0,1] + 0.4683[1,1] = [0.9366, 0.5317]$.
2. $Q : 10 \times 16$, $A : 10 \times 10$, head out 10×16, concat 10×64, $W^O : 64 \times 64$.
3. Unscaled scores row $2 = (1, 1, -\infty)$, so $A_2 = (0.5, 0.5, 0)$ and the output is $0.5[1,0] + 0.5[0,2] = [0.5, 1]$.